4. 실험

4.1 실험 설정

8개 리전의 2025년 시간별 탄소집약도는 Electricity Maps[31]에서 받았다. 값은 수입과 수출을 반영한 소비 기준의 평균 배출계수다[32]. LSTM의 예측이 1월 8일부터 나오므로 1월 1~7일은 준비 구간으로 두고, 모든 지표는 1월 8일부터 연말까지 8,592시간 동안 센다. 3.2절의 세 리전(캘리포니아, 텍사스, 독일)이 받는 앞으로 24시간의 기상값은 실제 관측값이다.

워크로드는 두 가지다. 합성 워크로드는 리전마다 하루 50건씩 1년 146,000건이며, 채점 구간에 제출된 작업은 143,200건이다. 작업은 허용 대기 시간에 따라 다섯 등급으로 나누고(표 3), 등급 안에서 허용 대기 시간은 로그균등 분포, 실행 시간은 로그정규 분포에서 뽑는다. Azure 워크로드는 Azure의 2019년 VM 기록[34, 35]에서 VM 146,016개를 뽑아 1년으로 이은 것이다. 도착 시각과 실행 시간은 기록 그대로이고, 제출 리전은 여덟 리전에 고르게 나누며, 허용 대기 시간은 합성과 같은 등급 비율로 준다. 실행 시간의 중앙값은 합성 81초, Azure 1,500초다. 연기 불가 작업이 전체 실행 시간에서 차지하는 비율은 합성 1.3%, Azure 51%다.

모든 작업을 제출 리전에서 즉시 실행할 때 한 리전이 동시에 실행하는 작업 수의 최댓값을 기준 피크라 한다. 기준 피크는 합성 13, Azure 34다. 리전 상한은 기준 피크의 1.25배를 올림한 값으로 모든 리전에 같게 주며, 합성 17, Azure 43이다. 예비 용량은 합성 3, Azure 0이다. 상한을 바꾼 결과는 4.6절에 있다.

예측기는 3.2절의 LSTM을 기본으로 쓰고, 24시간 전 같은 시각의 값을 쓰는 예측과 실측값을 그대로 쓰는 완전예지를 함께 쓴다.

비교 대상은 2장의 선행 연구에서 가져온다. CICS[3], CASPER[4], Sukprasert 등[6], Caspian[8]의 정책은 원문의 수식대로 다시 구현해 같은 자료와 같은 회계로 채점한다. 도착 시 배정 방식[9]은 작업이 도착한 슬롯에서 이미 정한 배정을 모두 보고 리전과 시작 시각을 한 번 정한 뒤 다시 풀지 않는다. 이 방식은 CAST와 같은 선형 계획, 예측, 비용 위에 구현해 확정 시점만 다르게 했다. 표 4는 각 정책의 재현 설정이다.

지표는 네 가지다. 탄소 절감률은 모든 작업을 제출 리전에서 즉시 실행할 때(기준)의 배출 대비 줄어든 비율이다. 상한 위반은 이미 상한만큼 찬 리전에서 시작한 작업 수다. 왕복 지연 시간은 실행 시간으로 가중한 평균(ms)이고, 대기 시간은 제출에서 시작까지의 평균(시간)이다. 선행 정책 비교에는 그 정책이 위반 없이 실행되려면 한 리전에 필요한 동시 실행 수(필요 용량)를 더한다. 두 실행의 배출을 비교할 때는 왕복 지연 시간이 0.6 ms 안에서 같아지도록 한쪽의 α를 맞추고, 7일 단위로 묶어 2,000번 다시 뽑는 블록 부트스트랩으로 95% 신뢰구간을 구한다.

표 3 합성 워크로드의 등급

Table 3 Job classes of the synthetic workload

Class	Allowed wait	Median run (sec)	Jobs
1	12–24 h	5,400	43,830
2	2–8 h	1,800	14,458
3	2–10 min	180	14,716
4	15–60 s	30	29,244
5	1–5 s	5	43,752

표 4 비교 대상과 재현 설정. Both는 리전과 시각을 모두 옮긴다는 뜻이고, Sukprasert는 1년치 실측값을 미리 안다.

Table 4 Compared policies and their settings. Both: region and time; Sukprasert knows the whole year in advance.

Policy	Shifts	Capacity	Deadline	Carbon
CICS	Time	Daily curve	Kept	Actual
CASPER	Region	Own limit	Kept	Actual
Sukprasert	Both	None	Kept	Actual
Caspian	Both	Hard 17	Soft	LSTM
Arrival-time	Both	All plans	Kept	LSTM

4.2 주요 결과

CAST는 두 워크로드 모두 상한 위반 없이 배출을 줄였다(표 5). 무릎점 규칙으로 합성에서 71.93%, Azure에서 51.35%를 줄였고, 탄소만 고려하는 α=1에서는 각각 95.65%, 67.74%를 줄였다. 합성에서 시간 이동만 허용하면 18.78%, 리전 이동만 허용하면 61.57%이므로, 큰 절감은 리전 이동에서 나오고 시간 이동이 그 위에 더해진다. Azure는 연기 불가 작업이 실행 시간의 절반을 차지해 시간 이동만으로는 1.53%를 줄인다.

표 5 주요 결과. 별도 표시가 없으면 예측기는 LSTM, 가중치는 무릎점 규칙이다.

Table 5 Main results. LSTM forecasts and the knee rule unless noted.

Workload	Setting	Savings (%)	Viol.	RTT (ms)	Wait (hr)
Synthetic	CAST	71.93	0	47.5	1.47
Synthetic	CAST, α=1	95.65	0	121.3	2.42
Synthetic	Time only	18.78	0	0	2.17
Synthetic	Region only	61.57	0	47.3	0
Synthetic	CAST, SN-24h	64.44	0	36.7	2.45
Synthetic	CAST, oracle	67.20	0	38.8	2.13
Azure	CAST	51.35	0	39.4	2.25
Azure	CAST, α=1	67.74	0	115.8	3.58
Azure	Time only	1.53	0	0	2.10
Azure	Region only	52.10	0	46.5	0

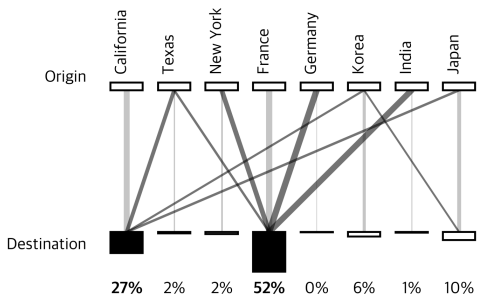


그림 4 리전 간 작업 이동(합성, 무릎점, 건수 기준)

Fig. 4 Job flows between regions (synthetic, knee, by job count)

그림 4는 무릎점 규칙에서 작업이 옮겨 간 흐름이다. 작업의 58.4%가 제출 리전 밖에서 실행되었고, 실행된 작업의 52%가 프랑스, 27%가 캘리포니아에서 돌았다. 독일과 인도의 작업은 주로 프랑스로, 텍사스, 일본, 한국의 작업은 주로 캘리포니아로 간다.

4.3 선행 정책과의 비교

표 6과 그림 5는 선행 정책을 같은 자료와 회계로 재현한 결과다. 상한을 두지 않는 정책은 상한 17에서의 위반과 함께 필요 용량으로 비교한다.

같은 상한에서는 CAST가 더 많이 줄인다. CASPER에 상한 17을 주면 78.53%를 줄이고, 왕복 지연 시간이 비슷한 CAST(α=1)는 95.65%를 줄인다(117 ms 대 121 ms). Caspian은 상한 17을 지키는 대신 마감을 폐널티로 두어 5,582건(3.90%)이 마감을 넘기며 67.54%를 줄인다. CAST는 무릎점 규칙에서 마감 위반 없이 71.93%를 줄이고, 왕복 지연 시간도 짧다(47 ms 대 85 ms).

상한이 없으면 선행 정책도 큰 절감을 내지만, 한 리전에 큰 용량이 필요하다. CASPER는 95.30%, Sukprasert의 리전·시간 이동은 96.63%를 줄이며, 필요 용량은 각각 29와 129로 기준 피크 13의 2.2배와 9.9배다. Sukprasert의 정책은 1년치 탄소집약도를 미리 안다고 보며, 작업의 99.75%를 프랑스에서 실행한다. CAST(α=1)는 상한 17 안에서 95.65%를 줄인다. 제출 리전에서 시각만 옮기는 정책끼리 보면 CICS는 13.89%, Sukprasert의 시간 이동은 22.22%(필요 용량 27)를 줄이고, CAST의 시간 이동만은 18.78%를 상한 17 안에서 줄인다. 리전별 최대 동시 실행을 모두 더한 설비 총량은 CASPER와 CAST(α=1)가 46으로 같고, CAST 무릎점은 117이다.

표 6 선행 정책과의 비교(합성). 필요 용량은 위반 없이 실행되는 데 필요한 리전 상한이다.

Table 6 Comparison with prior policies (synthetic). Req. cap. is the per-region ceiling needed to run without violations.

Policy	Savings (%)	Viol.	Req. cap.	Deadline miss	RTT (ms)
CICS	13.89	0	12	0	0
Sukprasert, time	22.22	3,180	27	0	0
CASPER	95.30	11,061	29	0	121.3
CASPER, cap 17	78.53	0	16	0	117.4
Sukprasert, region+time	96.63	61,539	129	0	121.3
Caspian	67.54	1,538	34	5,582	84.5
CAST	71.93	0	17	0	47.5
CAST, α=1	95.65	0	17	0	121.3

예측기를 바꾸어도 위반은 0이다. 평균 절대 오차는 LSTM 26.10, 24시간 전 값 25.27 gCO_2/kWh로 비슷하지만, 무릎점 절감은 LSTM 71.93%, 24시간 전 값 64.44%, 완전예지 67.20%다. 이 차이는 예측기마다 무릎점 규칙이 고른 α가 달라서 생긴다. α=1에서는 세 예측기가 95.59~95.87%로 0.3%p 안에 있다. 계절이 다른 네 주에서 미래를 모두 안다고 보고 한 주 전체를 한 번에 푼 배출 하한과 비교하면, α=1의 CAST는 그 하한과 0.33~1.47%p 차이다.

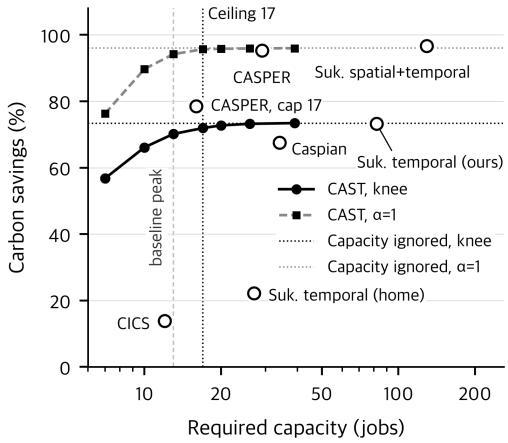


그림 5 탄소 절감률과 필요 용량. ○는 선행 정책, ●와 ■는 상한을 7~39로 바꾼 CAST이며, 점선은 상한이 없을 때의 CAST다.

Fig. 5 Carbon savings and required capacity. Hollow circles are prior policies, filled markers CAST with the ceiling swept from 7 to 39, and dotted lines CAST without a ceiling.

표 7 지연을 맞춘 도착 시 배정 방식 대비 CAST의 배출 차. 양수는 CAST가 덜 배출한 양(기준 배출 대비 %p)이다.

Table 7 Emission gap between the arrival-time method and CAST at equal latency. Positive values mean CAST emits less (pp of baseline).

Condition	Gap (pp)	95% CI
Synthetic, cap 17	0.96	0.51 to 1.46
Synthetic, 4 more draws	1.23 to 1.56	all above 0
Synthetic, cap 13	1.46	1.13 to 1.83
Synthetic, cap 26	0.34	−0.06 to 0.77
Synthetic, small caps in clean regions	2.10	1.83 to 2.36
Synthetic, oracle	1.01	0.80 to 1.25
Synthetic, no future information	0.96	0.74 to 1.18
Azure, cap 43	2.53	1.68 to 3.42

4.4 매시간 다시 푸는 효과

도착 시 배정 방식과 CAST는 같은 선형 계획, 예측, 비용을 쓰고 배정을 확정하는 시점만 다르다. 도착 시 배정 방식은 작업이 도착한 슬롯에서, CAST는 작업이 시작하는 슬롯에서 확정한다. 두 방식 모두 상한 위반이 없다. 이 절의 비교는 두 워크로드 모두 예비 용량 없이 한다.

왕복 지연 시간을 같게 맞추면 CAST가 덜 배출한다(표 7). 기준 배출 대비 합성에서 0.96%p, Azure에서 2.53%p 적고, 합성 워크로드를 다시 뽑은 네 벌에서도 1.23~1.56%p 적다. 깨끗한 리전의 상한이 작을수록 차이가 커서, 상한 13에서 1.46%p, 깨끗한 세 리전(캘리포니아, 프랑스, 뉴욕)의 상한을 작게 둔 배치에서 2.10%p다. 상한 26에서는 0.34%p로 신뢰구간이 0을 포함한다. 완전예지에서도 1.01%p, 미래 정보를 모두 뺀 설정에서도 0.96%p 차이가 난다. 확정을 시작 시점까지 미루므로 그사이 도착한 작업과 함께 자리를 다시 나눌 수 있기 때문이다. 대기 시간은 합성에서 CAST가 짧고(1.72 대 2.03시간), Azure에서는 길다(2.51 대 1.81시간).

확정한 계획을 서로 보는 것이 위반을 막는다. 도착 시 배정 방식에서 다른 작업의 확정 계획을 보지 않고 이미 시작한 작업만 보도록 바꾸면, 합성에서 작업의 9.0~17.0%, Azure에서 11.4~14.7%가 상한을 넘겨 시작한다. 상한을 13, 26, 리전별 상한으로 바꾸어도 12.03%, 4.76%, 14.97%가 상한을 넘는다. 같은 조건에서 CAST의 위반은 모두 0이다.

그림 6은 캘리포니아의 7월 7일이다. 탄소가 낮은 현지 9~16시에 CAST의 동시 실행이 상한 17에 닿아 있다. CAST는 이 시간을 상한까지 채우고, 나머지 작업은 그 앞뒤 시각에 시작한다.

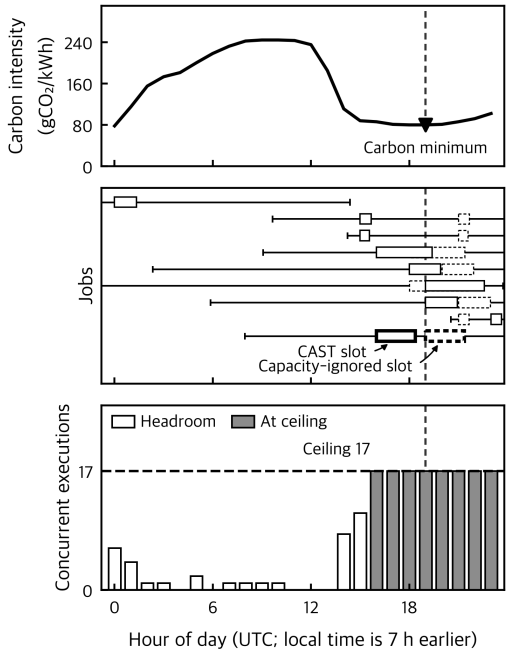


그림 6 캘리포니아의 하루(7월 7일). 가운데의 실선은 CAST가 정한 실행 구간, 파선은 상한이 없을 때의 실행 구간이다.

Fig. 6 One day in California (7 July). In the middle panel, solid boxes are CAST's runs and dashed boxes the runs without a ceiling.

표 8 α에 따른 배정(합성). FR(프랑스), CA(캘리포니아), Moved(옮긴 작업)는 실행 시간 기준 비율(%)이다.

Table 8 Assignment by α (synthetic). FR (France), CA (California) and Moved are shares of execution time (%).

α	Savings (%)	RTT (ms)	Wait (hr)	FR	CA	Moved
0.0	0.00	0.0	0.00	12.3	12.7	0.0
0.1	23.48	0.5	1.89	16.8	12.7	4.5
0.2	26.61	1.2	1.90	22.5	12.7	10.2
0.3	29.25	2.3	1.95	23.9	15.0	14.0
0.4	38.64	9.1	1.92	27.7	20.0	24.3
0.5	62.99	32.0	1.73	43.2	24.7	47.5
0.6	77.69	53.7	1.58	53.4	32.9	64.7
0.7	88.99	78.7	1.52	65.2	33.6	75.4
0.8	93.51	98.2	1.74	81.1	18.9	80.6
0.9	95.24	113.7	2.11	93.9	6.1	85.1
1.0	95.65	121.3	2.42	99.9	0.1	87.7
Knee	71.93	47.5	1.47	49.2	31.2	59.0

4.5 가중치 α에 따른 배정

표 8은 α를 고정한 CAST와 무릎점 규칙의 결과다(합성). α가 커질수록 작업이 깨끗한 리전으로 옮겨 간다. α=0.4까지는 옮긴 작업이 실행 시간의 24% 이하이고 절감은 대부분 시간 이동에서 나온다. α=0.5~0.7에서 작업이 프랑스와 캘리포니아로 옮겨 가며 절감이 63%에서 89%로 오르고, α=1에서는 실행 시간의 99.9%가 프랑스에서 돌며 왕복 지연 시간이 121 ms가 된다.

무릎점 규칙은 슬롯의 89%에서 0.5 또는 0.6을 골라, 실행 시간의 49%를 프랑스, 31%를 캘리포니아로 보낸다. 대기 시간은 1.47시간으로, 작업을 옮기는 고정 α 열 개보다 짧다. 왕복 지연 시간이 같은 고정 α(0.575)는 73.79%를 줄이고 대기 시간은 1.63시간이다. 같은 비교에서 고정 α는 합성 아홉 조건 가운데 여덟에서 1.77~2.64%p 더 줄였고, Azure에서는 차이가 없었다(0.63%p, 신뢰구간이 0을 포함). 대기 시간은 모든 조건에서 무릎점 규칙이 짧았다. 첫 주 자료로 정한 고정 α(0.5687)는 1년 동안 72.86%를 줄였다.

4.6 민감도 분석

표 9는 조건을 하나씩 바꾼 결과다. 합성 실행은 모두 상한 위반이 0이다. 상한을 기준 피크의 0.5배(7)에서 3배(39)로 늘리면 무릎점 절감은 56.79%에서 73.42%로 오르고, 2배(26) 근처에서 거의 멈춘다. 상한 7에서는 모든 작업을 제출 리전에서 즉시 실행해도 상한을 713번 넘지만, CAST는 위반 없이 56.79%를 줄인다. 리전마다 상한을 달리 준 세 배치에서는 63.75~73.05%를 줄인다. 작업을 다시 뽑은 다섯 벌에서 무릎점 절감은 71.51±0.43%이고, 도착을 두 배로 늘리면 69.45%다. 예측 구간을 6~12시간으로 줄이면 71.04~71.63%이고, 미래 정보(슬롯 안 도착, 이번 슬롯의 실측 탄소집약도, 기상 관측값)를 모두 빼면 68.88%다.

Azure에서는 상한을 기준 피크의 0.6배(21)로 줄여도 위반이 없다. 0.5배(17)에서는 465건(0.32%)이 상한을 넘겨 시작했고, 모두 다른 리전도 전부 상한까지 찬 순간이었다. 같은 상한에서 도착 시 배정 방식은 847건(0.59%)이다.

표 9 조건을 바꾼 결과. 별도 표시가 없으면 합성 워크로드, LSTM, 상한 17이다.

Table 9 Results under changed conditions. Synthetic workload, LSTM and cap 17 unless noted.

Condition	Knee (%)	α=1 (%)	Viol.
Main	71.93	95.65	0
Forecaster: SN-24h	64.44	95.59	0
Forecaster: oracle	67.20	95.87	0
5 workload draws	71.51 ± 0.43	95.40 ± 0.28	0
Cap 7 (0.5×)	56.79	76.30	0
Cap 13 (1×)	70.16	94.18	0
Cap 26 (2×)	73.21	95.90	0
Cap 39 (3×)	73.42	95.96	0
Per-region caps, 3 layouts	63.75–73.05	87.38–95.77	0
Arrivals ×2 (cap 23)	69.45	92.58	0
Deferrable share 20–60%	70.01–72.54	93.35–95.70	0
Horizon 6–12 h	71.04–71.63	95.58–95.65	0
No future information	68.88	—	0
Azure, cap 0.5× (17)	19.70	—	465
Azure, cap 0.6× (21)	27.17	—	0
Azure, cap 0.75–2×	34.15–61.89	—	0

표 10 회계 방식별 CAST와 도착 시 배정 방식의 배출 차(%p). 양수는 CAST가 덜 배출한 양이다.

Table 10 Emission gap between the arrival-time method and CAST by accounting (pp). Positive values mean CAST emits less.

Accounting	Synthetic	Azure
Average factor	0.96	2.53
Exact overlap	0.88	2.52
Transfer 1 GB, destination	1.03	2.50
Transfer 1 GB, origin	1.01	2.59
Life-cycle approx.	0.95	2.40
PUE 1.6 at FR, CA	1.07	2.32
PUE 1.1 at FR, CA	0.89	2.68
Marginal, France only	−0.89	1.48
Marginal, symmetric	1.19	2.05
France only + origin 1 GB + PUE 1.6	−1.49	0.89

표 10은 회계 방식을 바꾸어 표 7의 첫 비교를 다시 센 결과다. 배정은 그대로 두고 배출만 다시 셌다. 실행이 슬롯과 겹친 길이로 정확히 적분하거나, 옮긴 작업마다 1 GB의 전송 에너지(0.06 kWh[37])를 더하거나, 무탄소 전원에 전과정 배출(원자력 12, 재생 30 gCO_2/kWh)을 더하거나, 리전 설비 효율(PUE)을 달리 두어도 CAST가 합성에서 0.88~1.07%p, Azure에서 2.32~2.68%p 덜 배출한다. 프랑스로 옮긴 작업을 같은 시각 독일의 탄소집약도로 세는 한계 배출 근사에서는 합성에서 도착 시 배정 방식이 0.89%p 덜 배출하고, Azure에서는 CAST가 1.48%p 덜 배출한다. 출발 리전으로 센 전송 에너지에서는 옮긴 작업마다 약 9 GB를 보내면 CAST 무릎점의 절감이 시간 이동만(18.78%)과 같아진다.

4.7 계산 비용

CAST는 매 슬롯 후보 α 11개마다 선형 계획 문제를 푼다. SciPy 1.18.0의 HiGHS로 Apple M1 Max(메모리 32 GB)에서 돌렸을 때, 합성 워크로드의 슬롯당 풀이 시간은 평균 223 ms, 95 백분위 380 ms, 최대 1,223 ms이고 1년 전체에 34.4분이 걸렸다. 한 슬롯 문제의 변수는 가장 많을 때 7,848개였다. Azure에서는 슬롯당 평균 318 ms, 1년 49.3분이 걸렸고, 합성의 도착을 두 배로 늘리면 변수가 15,816개, 슬롯당 평균 556 ms다. 모두 슬롯 길이 1시간보다 훨씬 짧다.